

Create Internal Task

Purpose

The Create Internal Task feature allows users to create a new **Internal Task** from supported source records through a command bar button.

Supported source records:

- **contact**
- **account**
- **wrmb_portfolio**

The feature opens a dialog where the user selects an allowed Internal Task Type. After selection, a new unsaved **Internal Task** create form is opened with predefined lookup values. The record is not saved automatically.

User Flow

1. The user opens a supported source record.
2. The user clicks the command bar button **Create Internal Task**.
3. A dialog WebResource opens.
4. The dialog shows only task types that are:
 - enabled in configuration,
 - allowed for the current source entity,
 - allowed for the current user's security roles.
5. The user selects one task type and clicks **Create**.
6. Dynamics opens a new **Internal Task** create form.
7. The form receives default lookup values:
 - source lookup, depending on the original record,
 - Internal Task Type lookup.
8. The existing legacy Internal Task form script runs and applies its existing create behavior.

Main Components

Component	Purpose
<code>createInternalTaskCommand.js</code>	Command bar entry point and Internal Task form wrapper.
<code>createInternalTaskDialog.html</code>	Dialog UI WebResource.
<code>createInternalTaskDialog.js</code>	Dialog logic and form opening.
<code>nev_config</code>	Dataverse configuration table.
<code>nev_internals tasktype</code>	Table used to resolve the selected Internal Task Type.
Legacy <code>nev_/InternalTasksFunctions.js</code>	Existing Internal Task form behavior.

Dataverse Configuration

The feature reads its configuration from table `nev_config`.

Required config record:

Field	Value
<code>nev_key</code>	<code>idInternalTaskDialogConfig</code>
<code>nev_value_ntext</code>	JSON configuration

Web API calls must use the lowercase logical / OData names:

- `nev_key`
- `nev_value_ntext`

Do not use schema-name casing such as `nev_Key` or `nev_Value_nText` in Web API queries.

JSON Configuration

Example:

```
{
  "version": 1,
  "taskTypes": [
    {
      "key": "review",
      "label": "KYC Review",
      "taskTypeCodeName": "KYC_PERIODIC_REVIEW",
      "allowedRoles": ["WRM Internal Task Create"],
      "sourceEntities": ["contact", "account", "wrmb_portfolio"],
      "enabled": true
    }
  ]
}
```

JSON Properties

Property	Required	Meaning
<code>version</code>	Yes	Configuration schema version. Currently 1.
<code>taskTypes</code>	Yes	List of selectable task types.
<code>key</code>	Yes	Stable technical identifier for this menu option.
<code>label</code>	Yes	Text shown in the dialog dropdown.
<code>taskTypeCodeName</code>	Yes	Value matched against <code>nev_internaltasktype.nev_internaltasktypecodename</code> .
<code>allowedRoles</code>	No	Security role names allowed to use this option. Empty or missing means all users are allowed for this option.

Property	Required	Meaning
<code>sourceEntities</code>	No	Source entity logical names where this option is available. Empty or missing means all supported source entities.
<code>enabled</code>	No	<code>false</code> disables the option. Missing or <code>true</code> means enabled.

Security Role Behavior

Security is evaluated per configured task type.

The current user's role names are read from:

```
Xrm.Utility.getGlobalContext().userSettings.roles
```

The dialog option is visible if at least one role in `allowedRoles` matches the current user's roles.

Rules:

- `allowedRoles: ["Role A"]` means only users with `Role A` can see and create that option.
- `allowedRoles: []` means every user can see and create that option.
- Missing `allowedRoles` also means every user can see and create that option.

Recommended setup:

- Use dedicated business roles such as `WRM Internal Task Create` or `WRM Internal Task Approval`.
- Keep role names stable, because the configuration compares by role name.
- Use Ribbon Workbench display rules or privilege rules for coarse button visibility.
- Use JSON `allowedRoles` for option-level visibility inside the dialog.

Source Lookup Mapping

The created Internal Task receives one source lookup depending on where the user started:

Source entity	Internal Task lookup
<code>contact</code>	<code>nev_contactid</code>
<code>account</code>	<code>nev_companyid</code>
<code>wrmb_portfolio</code>	<code>nev_portfolioid</code>

The Internal Task Type lookup is set on:

```
nev_internaltasktype
```

The selected task type is resolved by:

```
nev_internalsertype.nev_internalsertypecodename = taskTypeCodeName
```

Form Integration

The new create form is opened with:

```
Xrm.Navigation.openForm
```

The record is opened as an unsaved create form. The solution does not call `createRecord` and does not save the Internal Task automatically.

This is intentional because the existing Internal Task form script must still run its standard create behavior.

Legacy Script Integration

The Internal Task form uses a wrapper handler:

```
WRM.createInternalTaskCommand.initializeCreateForm
```

This wrapper:

1. Checks if the form is Create or Quick Create.
2. Reads create-form parameters and `extraqs`.
3. Sets empty lookup fields from parameters.
4. Calls the existing legacy `OnLoad` once.
5. If `nev_internalsertype` has a value on create, triggers the normal Dynamics `fireOnChange`.

The last step is required because the legacy script registers the `nev_internalsertype` `OnChange` handler during `OnLoad`. The `OnChange` then fills fields such as Approval Flow Type and Flow Step according to existing legacy behavior.

Required Dynamics Form Setup

On the `Internal Task` main form:

1. Add the legacy `nev_/InternalTasksFunctions.js` WebResource as a form library.
2. Add `mhwrmb_createInternalTaskCommand.js` as a form library.
3. Ensure the legacy library is loaded before the wrapper library.
4. Register only this `OnLoad` handler:

```
WRM.createInternalTaskCommand.initializeCreateForm
```

5. Enable `Pass execution context as first parameter`.

6. Do not register the old legacy `OnLoad` as a separate form event handler.

The old legacy library must still be loaded, but its `OnLoad` is called by the wrapper. Registering both handlers separately would run the legacy logic twice.

Required Command Bar Setup

Create a command bar button on the supported source forms.

Recommended button label:

```
Create Internal Task
```

Command action:

```
WRM.createInternalTaskCommand.openDialog
```

Parameter:

```
PrimaryControl
```

For basic command availability, the bundle also exposes:

```
WRM.createInternalTaskCommand.canCreateInternalTask
```

This currently returns `true`. Detailed option-level security is handled by the JSON configuration and dialog filtering. If the button itself must be hidden for users without general access, configure Ribbon Workbench display rules or privilege rules.

Required WebResources

The following WebResources must be deployed:

WebResource	Purpose
<code>mhwrm_createInternalTaskCommand.js</code>	Command and form wrapper logic.
<code>mhwrm_createInternalTaskDialog.html</code>	Dialog HTML.
<code>mhwrm_createInternalTaskDialog.js</code>	Dialog JavaScript.

The HTML references the dialog JavaScript as:

```
<script src="mhwrmb_createInternalTaskDialog.js"></script>
```

Both dialog WebResources must therefore exist with matching names in Dynamics.

Supported Creation Paths

The solution supports three creation paths:

- Manual Internal Task creation.
- Create Internal Task command button and dialog.
- Related-record or subgrid creation.

The wrapper only fills empty fields. Existing values provided by Dynamics relationship mapping or user input are not overwritten.

Troubleshooting

The button opens, but the dialog shows no options

Check:

- `nev_config` contains a record with `nev_key = idInternalTaskDialogConfig`.
- `nev_value_ntext` contains valid JSON.
- The selected source entity is listed in `sourceEntities`.
- The task type entry is not disabled with `"enabled": false`.
- The current user has at least one role listed in `allowedRoles`, unless `allowedRoles` is empty.

The user gets a permission message

Check `allowedRoles` in the JSON. If the array is empty, the option is available for all users. If role names are configured, they must exactly match the Dynamics security role names.

The Internal Task Type is not found

Check that `taskTypeCodeName` in JSON matches:

```
nev_internalsubtasktype.nev_internalsubtasktypecodename
```

Approval Flow Type or Flow Step are not set

Check:

- `nev_/InternalTasksFunctions.js` is loaded as a form library.
- `WRM.createInternalTaskCommand.initializeCreateForm` is the only OnLoad handler.
- `Pass execution context` is enabled.
- `nev_internalsubtasktype`, `nev_approvalflowtype`, and `nev_flowstep` are available on the form.
- Browser and Dynamics WebResource cache were refreshed after deployment.

Deployment Notes

Build the WebResources with:

```
npm run build
```

Deploy the generated JavaScript from **Deployment** or the agreed solution packaging process.

For production, deploy JavaScript WebResources without inline source maps. Separate **.map** files may be deployed to test environments only if debugging is required.